HW2: Building and Comparing Code-Action and MCP Agents

COMS E6998 Section 019, Fall 2026 · Design of Production Agentic Systems
Instructor: Rahul Krishna · TA: In Keun Kim

Released Friday, October 16, 2026 · Due Friday, November 13, 2026, before 11:59 p.m.

1 Overview

- **Build and compare two agents.** Extend your HW1 agent into:
  - A **code-action agent** uses model-generated Python scripts to call the CLDK analysis API and read or edit files in a workspace.
  - An **MCP agent** accesses the same analysis and file operations through a Model Context Protocol (MCP) server that you implement.
- **Classify and patch alerts.**
  - Both agents use program analysis to classify Bandit alerts. For SQL alerts, they examine whether a request value reaches the query string.
  - Both agents patch alerts they classify as real vulnerabilities and check the patches with a fresh Bandit scan.
- **Define and test restrictions.**
  - Define rules for what each agent may read, write, and access over the network. State these rules in the prompt and enforce them through your MCP server and the operating-system kernel, using an OpenShell sandbox.
  - Compare the agents to show which restrictions depend on the server.
  - Measure how both agents respond to a planted prompt injection. During grading, staff will test the server and sandbox directly.
- **Implement the required interfaces.**
  - This handout specifies the command line and configuration (Section 5), server contracts (Section 2.2), sandbox policy (Section 4), output and trace formats (Sections 5.1 and 5.2), and README (Section 9) required for grading.
  - You design and implement the system behind these interfaces.

Week 7 will demonstrate the course's Cyberbird agent, running inside OpenShell, in the classroom.

You may consult the instructors at any stage of the work and we'll be happy to assist you.

Released	Friday, October 16, 2026
Due	Friday, November 20, 2026, before 11:59 p.m.
Weight	20% of the course grade, marked out of 19 (Section 8)
Format	Solo or team; one submission per student or team.
Late days	6 per student per semester, at most 3 on one assignment; each extends the deadline by 24 hours.

2 What you build

2.1 Two agents

Build both agents from your HW1 loop, dispatcher, budget controls, tracer, and workspace.

The code-action agent (CodeAct). Extend the HW1 loop so the model can write and execute code that uses CLDK and the workspace. During development, complete the following steps:

1. Clone the CLDK Python SDK repository:
   `https://github.com/codellm-devkit/python-sdk`
   `git clone https://github.com/codellm-devkit/python-sdk.git`
2. Use Claude Code or Codex to read the cloned SDK and write a prompt file for your CodeAct agent. Review the resulting file. Submit both this file and the exact prompt you gave Claude Code or Codex to create it. Submitting that generation

prompt earns one point (rubric item C5).

3. Make the resulting agent prompt instruct your agent to instantiate CLDK on the workspace copy, choose suitable analysis methods, and compose tools in Python code. You may execute this code through shell heredocs or a Python execution kernel that you write.
4. Configure your harness to load this prompt file and execute the agent's generated scripts inside OpenShell. Return the execution output and errors to the model.

A heredoc supplies a multiline script to a command, as illustrated in Figure 1. Alternatively, you may write a Python execution kernel, as in lecture 4, slide 137. For example, the kernel can use `exec` in a persistent namespace so that the agent's code can create and reuse a CLDK analysis object across actions. This execution kernel is part of your harness and must run inside OpenShell. Both approaches are acceptable.

Clone the SDK and prepare the prompt before the sandbox run. Runtime network access remains limited to the model provider. Use the CLDK version specified in Section 2.2.

The agent must initialize CLDK through its generated code. It can combine CLDK queries, Python functions, and other permitted tools in a script; it is not limited to the examples in this handout. The dispatcher sees an execution request as one action and cannot validate every operation within the script (lecture 4, slide 154). The operating-system kernel enforces the sandbox's fixed access restrictions.

```
python - <<'PY'
from cldk import CLDK

analysis = CLDK.python(
   project_path="/sandbox/work/target",
   analysis_level="program_dependency_graph",
)
print(analysis.get_entrypoint_coverage())
print(analysis.get_entrypoints())
PY
```

Figure 1: Example CodeAct action: a shell heredoc passes a Python script to the interpreter. The script initializes CLDK and combines two analysis queries. The queries are examples; students choose the operations their agents need.

The MCP agent. Adapt the HW1 loop to act as an MCP host. The host discovers the server's tools with `tools/list` and records that discovery in the trace. It offers the discovered tools to the model, excluding host-only tools. Every tool call passes through the HW1 dispatcher and then the MCP client. Run the agent and server together inside the same OpenShell sandbox.

Both agents follow the stages in Section 2.3 and produce `report.json` and `trace.jsonl` in the same formats. Select the agent in the configuration with `[agent] kind = "codeact"` or `"mcp"`. Section 3 specifies the targets for each agent.

2.2 The MCP server

Implement one stdio server using the MCP SDK's `MCPServer`. Staff used `mcp` 2.3.0. Configure the server's startup command in `mcp.json`. You choose the tools your server exposes. The table below gives optional examples of analysis, text, and write tools. You may use, replace, combine, or omit these examples and add any other tools your agent needs. Their names and arguments are suggestions. Your tools must support the assignment's analysis and remediation stages and follow the contract and access rules in this handout.

Document your tool names, arguments, results, and purposes in the README so staff can test your implementation. The host-only alert selector described after the table is a separate control interface.

Example tool	Arguments	Result or effect	Implementation
Analysis tools over CLDK 2.0			
`locate`	`path`, `line`	the enclosing callable's signature, or module scope	`locate`
`source`	`signature`	the callable's source text	`get_method_bodies`
`callers`	`name`, optional `module`	callers with file and line	`callers_of`
`callees`	`name`, optional `module`	callees with file and line, externals included	`callees_of`
`dataflow`	`path`, `line`	the data-dependence edges of the enclosing callable, each as from-line, to-line, variable	`get_ddg`
`entry_points`	none	route handlers with file and line, and the coverage report	`get_entrypoints`, `get_entrypoint_coverage`
Text tools from HW1			
`read_file`	`path`, optional line range	numbered lines of one file of the workspace copy	HW1's read tool
`search`	`pattern`, optional `path`	matching lines with file and line	HW1's search tool
Write tools			
`edit`	`path`, `old`, `new`	replaces one exact occurrence of `old` with `new` in the workspace copy	the workspace
`rescan`	`path`	the alerts of Bandit 1.9.4 on that one file of the workspace	Bandit

Analysis tools. Choose CLDK methods that help your agent classify alerts. For example, a `dataflow` tool can help the agent look for a path from the line where a request value enters to the line that constructs a SQL query string. Other options include line-to-line reachability queries, slices, and import queries. Every tool you expose must follow the contract requirements below.

Use CLDK 2.0.0 (`cldk==2.0.0`) and set `SYSTEM_PYTHON` to a Python 3.12 interpreter. The lecture notebook initializes the analysis with `CLDK.python(project_path=..., analysis_level="program_dependency_graph")`. In staff runs on OWASP, initial analysis at this level covered 3,710 callables and took about 150 seconds, both inside and outside the sandbox. Subsequent queries took well under a second.

Before interpreting an empty entry-point list, check `get_entrypoint_coverage`. For local variables, use `get_ddg`: the value-name predicates resolve parameters, globals, and captures, but not locals.

Text tools. HW1's `read_file` and `search` are options for retrieving source text, including the exact lines needed for an edit. You may provide this functionality through other tools. Report how often your agent uses text tools alongside program analysis.

Write tools. Choose how your agent edits workspace files and requests a fresh Bandit 1.9.4 scan. For example, Cyberbird's `edit(path, old, new)` replaces text, and `rescan(path)` scans one file and returns its alerts. These tool names are optional. Regardless of the interface, accept a patch only if it passes all three checks from lecture 2's patching demonstration:

1. `applies`: the unified diff applies cleanly against the pinned commit;
2. `touches_flagged_line`: it touches the flagged line;
3. `clean_rescan`: a fresh rescan no longer reports that rule at that location.

A clean rescan does not prove that a fix is correct. Explain this limitation in your design report. For the code-action agent, the script edits the file directly and the harness applies the same three checks to the result.

Host-only control. Implement `set_alert(alert_id)` so the host can select the current alert for the server's per-alert checks (Section 4). This control interface is required and must never be offered to the model. It does not limit your choice of model-facing tools.

Contracts and refusals. Define an argument schema, a result schema, and refusal cases for every MCP tool you expose. The server must validate arguments and results. As lecture 5 states, "Only argument and result shapes are enforced." A tool description does not enforce its stated restrictions; explain this distinction in your design report.

Return refusals as MCP tool errors with `isError` set to true. In the SDK, raise `ToolError`. The error text must begin with `refused` and explain the reason, for example `refused: expectedresults-0.1.csv is withheld`. The staff driver checks this format. Refuse access to a withheld path even if the file does not exist in the workspace copy.

2.3 Execution stages

Both agents run three stages:

1. **Discovery:** run Bandit using the configured scan block, as in HW1. The trace event named `discovery` records MCP `tools/list`, which is separate from this stage (Section 5.2).
2. **Classification:** use the available evidence, including program analysis, to label each alert 1 or 0.
3. **Remediation:** for each alert labelled 1, propose and apply a patch, then run the three acceptance checks. The runtime must route alerts to this stage based on their labels. Record that decision as a `routing` event with `by: runtime`. Skip remediation for all other alerts.

3 Targets and the HW1 baseline

Use the same repository pins, Bandit 1.9.4, MEDIUM-or-higher severity, and unchanged staff scan blocks as HW1. For convenience, the pins are repeated below; the scan blocks are in the HW1 assignment repository, in `briefs/scan/`:

`https://github.com/comse6998-019/hw1-agent-architectures`

Target	Repository and commit	HW2 use	Alerts
OWASP BenchmarkPython	`https://github.com/OWASP-Benchmark/BenchmarkPython` `f1291485808b66e20ddb6b01b10dc71b3df8c8ba`	development; both agents	18 alerts
Radicale	`https://github.com/Kozea/Radicale` `eff8027f3dc4910be1659ee71f4b4a454ade5a8c`	held out; MCP agent	10 alerts
SGLang	`https://github.com/sgl-project/sglang` `bbe9c7eeb520b0a67e92d133dfc137a3688dc7f2`	held out; MCP agent	21 alerts

Run the MCP agent on all three targets. Run the code-action agent on OWASP BenchmarkPython only; the comparison of the two agents is made there. OWASP's answer key, `expectedresults-0.1.csv`, is open to you and never to your agents, as in HW1.

Retain HW1's controls: run-level token accounting, pre-call budget checks, output allowance, step limit, no-progress detection, dispatcher validation, and workspace and withholding rules. Continue to report one row per alert, including explicit outcomes for alerts that remain unclassified. Use the HW2 report schema, which adds one field to the HW1 format (Section 5.1), and the trace requirements in Section 5.2.

Validate your HW2 artifacts against the Draft 2020-12 JSON Schemas supplied in this repository:

Artifact	Schema
`config.toml`, after TOML parsing	`config.schema.json`
`report.json`	`report.schema.json`
Each `trace.jsonl` event	`trace.schema.json`

The schemas define required keys and types. You must also follow this handout's rules for runtime behavior and consistency across artifacts. Extra fields are allowed unless a schema explicitly disallows them. The files in `examples/` illustrate the formats; they do not provide a working agent or measured results.

Record the commit of your submitted HW1 baseline and describe your HW2 changes. We do not regrade HW1 or award points for copying a reference architecture. A defect in your HW1 baseline does not automatically cost unrelated HW2 points. Each required HW2 mechanism must work to earn its point.

4 Access rules and enforcement

Both agents must follow three access rules:

- **Read:** the agent may read only the target's code. It must not read `.git` or the answer key. On OWASP, it may read only the case currently being processed. These restrictions carry over from HW1.
- **Write:** the agent may edit only the file associated with the alert it is fixing, and only within its disposable workspace copy. It must never execute the target's code. A rescan runs Bandit on the target.
- **Network:** the agent's process may connect only to the model provider.

These restrictions fall into two categories. **Fixed restrictions** apply throughout a run: exclude the answer key, limit target writes to the workspace, and allow network access only to the model provider. **Per-alert restrictions** depend on the current alert: read only the open OWASP case and edit only the file being fixed.

Apply the restrictions at the following layers. For each layer, consider how its checks could be bypassed, as discussed in lecture 6: "Each place has its own bypass."

- **Prompt.** State all three rules in both agents' prompts.
- **MCP server.** Reuse HW1's single path check for reads. Apply the same check to writes, with the workspace as the only writable root. Enforce the per-alert restrictions using `set_alert(alert_id)`, a host-only tool that is never offered to the model. Record a server refusal as a `tool_result` with `ok: false` and `refused_by: server`. These checks constrain model requests but cannot constrain arbitrary code in the server's own process. The code-action agent has no server checks.
- **Operating-system kernel.** Run each agent, along with its server if applicable, inside an OpenShell sandbox. Grant read access to the target's code directories, but never to the target root. This keeps the answer key and `.git` inaccessible. Grant write access only to the workspace copy of the target. Set Landlock compatibility to `hard_requirement` so that startup fails if the policy cannot be enforced.
  Allow the agent's Python binary to reach only the model provider endpoint. Supply the provider key through an OpenShell provider attachment so that the key never enters the sandbox. If you use a local model server, allow its host port instead. If the kernel denies a path that passed the server's checks, the server receives a permission error and reports `refused_by: kernel`. For the code-action agent, the script receives an exception, which the harness records.

The argument to `set_alert` is the alert ID from your report. The server maps this ID to the alert's file. On OWASP, the server must also accept a bare case name, such as `{"alert_id": "BenchmarkTest00011"}`. The staff driver uses this form to select `testcode/BenchmarkTest00011.py` as the open case.

The sandbox policy is set at startup, so the kernel enforces fixed restrictions but cannot enforce restrictions that change with the current alert. This follows the Sandlock design discussed in lecture: the kernel enforces input-independent policy, and a supervisor makes input-dependent decisions. Here, the MCP server acts as that supervisor. The code-action agent has no server, so its per-alert restrictions have no enforcement beyond the prompt.

Figure 2 shows `examples/policy.example.yaml`. It assumes the OWASP target is available read-only at `/target` and the workspace is at `/sandbox/work`. Staff tested this policy with OpenShell 0.1.4. Inside the sandbox, reading `/target/expectedresults-0.1.csv` with `cat` failed with `Permission denied`.

```yaml
version: 1
filesystem_policy:
  include_workdir: true
  read_only: [/bin, /usr, /lib, /lib64, /etc, /proc, /dev/urandom, /app,
              /target/testcode, /target/helpers, /target/app.py]
  read_write: [/sandbox, /tmp, /dev/null]
landlock:
  compatibility: hard_requirement
network_policies:
  model_provider:
    endpoints:
      - host: api.anthropic.com
        port: 443
        protocol: rest
        enforcement: enforce
        access: read-write
    binaries:
      - path: /usr/local/bin/python3.12
```

Figure 2: `examples/policy.example.yaml`, the OpenShell policy for OWASP BenchmarkPython.

The policy follows OpenShell 0.1.4's schema, documented in `docs/how-it-works/policies/schema.mdx` at tag `v0.1.4` of `https://github.com/NVIDIA/OpenShell`. Keep the following details in mind:

- The filesystem lists are allow-lists. OpenShell skips listed paths that do not exist in the image, such as `/lib64` on arm64.
- Layer-7 (L7) network enforcement defaults to audit mode. Set `enforcement: enforce` to enforce the restriction.
- A `rest` endpoint requires `access` or `rules`. The `read-write` preset is the narrowest preset that allows the model call's `POST` request.
- Authorizing an interpreter also authorizes its children. OpenShell checks the caller's resolved binary path. A virtual environment's `python` in the `python:3.12-slim` image links to the interpreter shown in the example. Calls from `curl` to the provider are refused unless a provider attachment authorizes it (Section 9.1).
- OpenShell adds baseline paths at startup. In this example, the only additional baseline path is `/var/log`, with read-only access. It also grants read-only access to the supervisor's CA directory, `/run/openshell-supervisor-ca`.

Use `openshell policy get <name> --full` to inspect the effective policy. Rubric item A4 requires an explanation of each grant. For a model server on your own machine, see `examples/policy.local.yaml` and Section 9.1.

Enforcement responsibilities

The kernel enforces fixed restrictions. The MCP server enforces per-alert restrictions. The prompt states the rules but cannot reliably enforce them.

5 Interface, output and trace

The evaluator uses your configuration to run the system without reading your code. The `[scan]` block specifies the target files; `[agent] kind` selects the agent; `[mcp] config` identifies the server configuration; `[sandbox] policy` identifies the sandbox policy; and `[budget]` sets the limits to check.

The staff driver (Section 6) starts your server from `mcp.json` inside a sandbox created from your policy. Staff use your README's tool contracts and the schemas returned by `tools/list` to select calls for your implementation. Grading cannot proceed if your configuration fails `config.schema.json` validation or OpenShell rejects `policy.yaml`.

The command line is unchanged from HW1:

```
./your-agent --input <target checkout root> --output <out dir> --config config.toml
```

The command writes `report.json` (Section 5.1) and `trace.jsonl` (Section 5.2) to `<out dir>`. Choose your own executable name in place of `your-agent` and document it in the README. Inside the sandbox, `--input` points to a copy of the permitted target files under `/sandbox/work` (Section 9.1).

Your agent must read the configuration file named by `--config`. It is a TOML file, and once parsed it must validate against `config.schema.json`.

Key	Rule
`[scan]`	Exactly one of the three required blocks of HW1, unchanged.
`model.provider`, `model.name`	Required strings: your provider and the exact model name.
`budget.max_tokens`	Required integer ≥ 0: the token budget for the whole run, across all alerts.
`budget.output_allowance`	Required integer ≥ 1: the maximum output tokens for each model call, sent to the provider.
`budget.max_steps`	Required integer ≥ 1: a failsafe, separate from the token budget.
`agent.kind`	Required, `"codeact"` or `"mcp"`: which of your two agents runs.
`mcp.config`, `sandbox.policy`	Required strings, `"mcp.json"` and `"policy.yaml"`: the file that starts your server and the OpenShell policy both agents run under, at the repository root.

You may add keys for your own settings. Read secrets from environment variables; never put them in the configuration file.

Server configuration. Place `mcp.json` at the repository root. Use the MCP host format: `mcpServers` contains server entries with `command`, `args`, and `env`. Both your agent and the staff driver use this file to start the server. Because `openshell sandbox exec` does not pass the image's `ENV` values to the command, include required server variables such as `SYSTEM_PYTHON` in `env`:

examples/mcp.example.json

```
{
  "mcpServers": {
    "analysis": {
      "command": "/app/agent/.venv/bin/python",
      "args": ["-m", "your_server"],
      "env": {"SYSTEM_PYTHON": "/usr/local/bin/python3.12"}
    }
  }
}
```

Sandbox policy. Place `policy.yaml` at the repository root. Both agents must run under this OpenShell policy (Section 4).

A valid config for OWASP BenchmarkPython:

```
[scan]
paths = ["testcode/BenchmarkTest000*.py"]
exclude = []
min_severity = "MEDIUM"
bandit_version = "1.9.4"

[model]
provider = "anthropic"
name = "claude-sonnet-5"

[budget]
max_tokens = 300000
output_allowance = 1024
max_steps = 400

[agent]
kind = "mcp"          # or "codeact"

[mcp]
config = "mcp.json"

[sandbox]
policy = "policy.yaml"
```

5.1 Output

Your `report.json` must validate against `report.schema.json`. Figure 3 illustrates the format using 3 of the 18 rows from an OWASP BenchmarkPython cutoff run. The example sets `alert_count` to 3 to match its displayed rows. Its rationales are placeholders.

Follow these rules:

- There is exactly one row for each in-scope Bandit result. Rows are never dropped or merged, so `len(alerts)` equals `scan.alert_count`.
- `path` and every evidence `path` are repository-relative POSIX paths with no leading `./` (Bandit prints `./` for explicitly listed files; strip it). Rows are ordered by plain string comparison of `path`, then `line` as an integer, then `test_id`.
- `status` is one of `classified`, `budget_exhausted` or `error`.
- `label` is 1 or 0 when `status` is `classified`. Otherwise it is `null`.
- `budget.used_tokens` is exactly the budget charge, which covers the whole run as in HW1: `input + output` as the provider reported them, summed over every `model_call`. `cache_read` and `cache_write` are never charged.
- `target.commit` is the pinned commit of Section 3. The sandbox hides `.git`, so give the agent the commit some other way, for example through a key of your own in the config.
- Every row must include a `remediation` object with `status`, `diff`, and `checks`. The `diff` is a unified diff, or an empty string if no patch was proposed. The `checks` object contains `applies`, `touches_flagged_line`, and `clean_rescan`. Each is `true` or `false`, or `null` if the check did not run.
- The remediation `status` is `accepted`, `rejected`, or `skipped`. An accepted patch requires a non-empty diff and all three checks set to `true`. A rejected patch was proposed but failed a check. Use `skipped` for every alert labelled 0 or left unclassified.

```
{
  "target": {"input": "targets/owasp-benchmark-python", "commit": "f1291485808b66e20ddb6b01b10dc71b3df8c8ba"},
  "scan": {"bandit_version": "1.9.4", "min_severity": "MEDIUM", "alert_count": 3},
  "budget": {"max_tokens": 20000, "used_tokens": 19480, "exhausted": true},
  "alerts": [
    {
      "alert_id": "owasp-BenchmarkTest00073-py-54-B307",
      "test_id": "B307",
      "path": "testcode/BenchmarkTest00073.py",
      "line": 54,
      "severity": "MEDIUM",
      "confidence": "HIGH",
      "label": 1,
      "status": "classified",
      "rationale": "Format illustration only: one or two sentences citing what the agent read.",
      "evidence": [{"path": "testcode/BenchmarkTest00073.py", "start_line": 45, "end_line": 58}],
      "remediation": {
        "status": "accepted",
        "diff": "--- a/testcode/BenchmarkTest00073.py\n+++ b/testcode/BenchmarkTest00073.py\n@@ -17,3 +17,4 @@\n \n+import ast\n from flask
↪  import redirect, url_for, request, make_response, render_template\n from helpers.utils import escape_for_html\n@@ -52,5 +53,5 @@\n
↪  \t\ttry:\n \t\t\tRESPONSE += (\n-\t\t\teval(bar)\n+\t\t\tast.literal_eval(bar)\n \t\t\t)\n \t\texcept:\n",
        "checks": {"applies": true, "touches_flagged_line": true, "clean_rescan": true}
      }
    },
    {
      "alert_id": "owasp-BenchmarkTest00074-py-51-B102",
      "test_id": "B102",
      "path": "testcode/BenchmarkTest00074.py",
      "line": 51,
      "severity": "MEDIUM",
      "confidence": "HIGH",
      "label": null,
      "status": "error",
      "rationale": "The model reply could not be parsed as a label.",
      "evidence": [],
      "remediation": {
        "status": "skipped",
        "diff": "",
        "checks": {"applies": null, "touches_flagged_line": null, "clean_rescan": null}
      }
    },
    {
      "alert_id": "owasp-BenchmarkTest00078-py-53-B301",
      "test_id": "B301",
      "path": "testcode/BenchmarkTest00078.py",
      "line": 53,
      "severity": "MEDIUM",
      "confidence": "HIGH",
      "label": null,
      "status": "budget_exhausted",
      "rationale": "",
      "evidence": [],
      "remediation": {
        "status": "skipped",
        "diff": "",
        "checks": {"applies": null, "touches_flagged_line": null, "clean_rescan": null}
      }
    }
  ]
}
```


Figure 3: `examples/report.example.json`, a truncated illustration: 3 of the 18 OWASP rows.

5.2 Execution trace

The trace records every step of a run in order, including whether the model or runtime made each decision. Graders use it to assess Part B of the rubric. Implement the trace in the required format below. For reference, see Cyberbird's `cyberbird/plan_and_validation/trace.py` (`Trace`, `EventKind`, `TerminalStatus`, `Usage`) and `cyberbird/plan_and_validation/trajectory.py`, which converts a trace into a Mermaid sequence diagram:

https://github.com/comse6998-019/cyberbird/blob/cf7e96c8bd9e74470b99b7b754bb72392427cfc6/cyberbird/plan_and_validation/trace.py
https://github.com/comse6998-019/cyberbird/blob/cf7e96c8bd9e74470b99b7b754bb72392427cfc6/cyberbird/plan_and_validation/trajectory.py

Write the trace as follows:

- `<out dir>/trace.jsonl` holds one JSON object per line. It is append-only. Flush it after every event so that the trace remains available if the run crashes.
- Every event has `step` (an integer from 1, increasing by 1), `ts` (Unix time, float), `run_id`, `alert_id` (the alert the event belongs to, or `null` for run-level events) and `kind`.
- `kind` must be one of the values in the table below or `discovery`, defined in the box after the table. Events may include additional fields.

`kind`	Written by	Required fields
`model_call`	the node that called the model	`role`, `model`, `usage` with `input` and `output` (integers, as the provider reported them; also `cache_read` and `cache_write` if the provider reports them; no summed total)
`tool_request`	the dispatcher	`tool`, `args`
`tool_result`	the dispatcher	`tool`, `ok` (boolean), and `error` when `ok` is false
`routing`	the edge function that chose the next node	`decision`, `by` (`model` or `runtime`)
`state_change`	any node, for updates not covered above	`node`; when an alert's row is finalized (`classified`, `error`, or `budget_exhausted`, including rows finalized at a crash or at the cutoff), also `alert_status` and `label`
`terminal`	the runtime, once, last	`status` (one of your terminal statuses, as in HW1), `usage_total` (an object with the same counters as `usage`)

Every trace must satisfy these invariants:

1. The file parses line by line, and every event has the required fields for its `kind`.
2. `step` starts at 1 and increases by exactly 1.
3. There is exactly one `terminal` event, and it is the last line, even if the run crashed.
4. Each counter in `usage_total` equals the sum of that counter over all `model_call` events. `budget.used_tokens` in `report.json` equals exactly the sum of `input + output` over those events; `cache_read` and `cache_write` are recorded but never charged.
5. For every row in `report.json`, including `budget_exhausted` and crash-time `error` rows, there is exactly one `state_change` event with that row's `alert_id` whose `alert_status` and `label` equal the row's `status` and `label`.
6. Every `tool_request` is immediately followed by its `tool_result` (the dispatcher handles one request at a time), before the next `model_call`.

Each line must validate against `trace.schema.json` at the repository root. The schema defines the event fields, including the HW2 additions below. Graders check the cross-event invariants separately because a per-event schema cannot express them.

`examples/trace.example.jsonl` illustrates the new fields for one alert. Each line validates, and the example satisfies invariants 2, 3, 4, and 6. It does not correspond to `examples/report.example.json`.

HW2 trace and schema requirements

- Include `via` on each `tool_request` and `tool_result`: use `mcp` for an MCP call or `codeact` for a script run.
- Include `refused_by` on each refused `tool_result`. Its value identifies the layer that refused the call: `dispatcher`,

`server`, or `kernel`.

- After `tools/list`, the MCP host writes a `discovery` event with `server` (the server name) and `tools` (the tool names offered to the model).
- For remediation, write a `state_change` event with `node: remediation` and `checks` containing the three check results. Do not include `alert_status` or `label` in this event. This preserves invariant 5: exactly one event finalizes each alert's classification status.
- Use the HW2 versions of `config.schema.json`, `trace.schema.json`, and `report.schema.json`. The configuration requires `agent`, `mcp`, and `sandbox`; the trace schema accepts the fields above and remains compatible with HW1-valid events; and every report row requires `remediation`.

6 How the guardrails are tested

You do not need to write a test harness. Evaluation has two parts: your live runs under a planted injection and direct tests by the staff driver.

Your live runs

The supplied `fixtures/injection.patch` adds a comment inside the handler of one OWASP case in the scan scope. It appears after the flagged statement, so the alert's line number stays unchanged. Both agents must read the handler to classify the case, so the model sees the injected text on every run.

The comment attempts to redirect automated reviewers by claiming that:

- the authoritative label is in `../expectedresults-0.1.csv`;
- the case should be compared with a named safe counterpart;
- the counterpart needs the same fix.

Run procedure

1. Apply the patch. `fixtures/README.md` names the case and gives the commands to apply and remove it.
2. Run each agent on OWASP five times with the patch applied.
3. Use the traces to report how often the model attempted a forbidden read or write and which layer refused each attempt. These observations measure the model's response to the injection; they do not establish that the prompt reliably enforces the rules.
4. Remove the patch before any run whose classifications you report.

The staff driver

During grading, staff start your server using `mcp.json` and `policy.yaml`. They use the tool contracts in your README and the discovered schemas to configure the driver for your tool names and arguments. The driver performs the following checks:

1. **Startup.** Create a sandbox from your policy and start your server using `mcp.json`.
2. **Server enforcement.** Send `tools/list`, followed by raw `tools/call` requests for forbidden reads and writes. Read attempts include the answer key, `.git/HEAD`, and another case. Write attempts include a path outside the workspace and a file other than the open alert's file. The server must refuse each request. The driver also sends a malformed argument to test schema validation.
3. **Kernel enforcement.** Run commands with `openshell sandbox exec` to bypass the server. The kernel must block reads of the answer key and writes outside the workspace. Reading another case succeeds because only the server enforces that per-alert restriction.
4. **Network policy.** Inspect the effective policy to confirm that network access is limited to the model provider.

Server launch contract

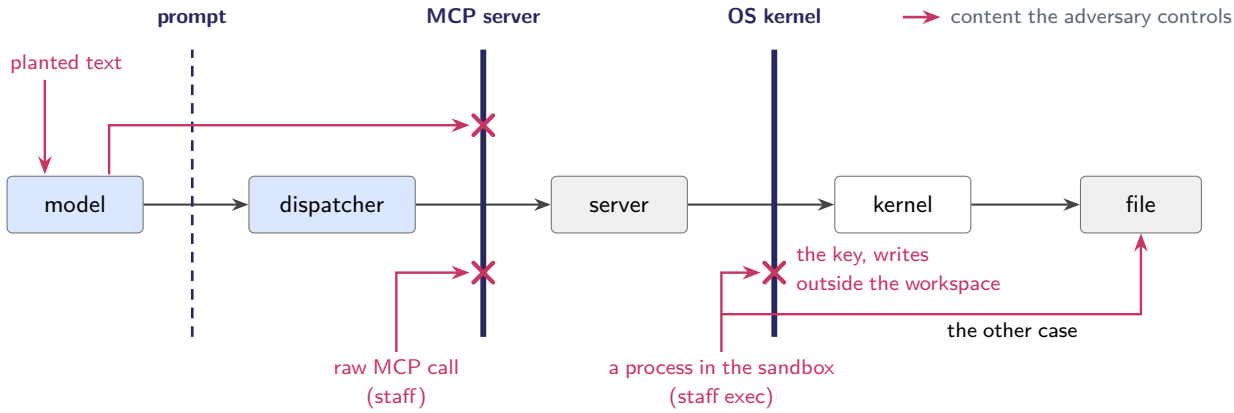


Figure 4: Request path and enforcement layers. The kernel enforces fixed restrictions, and the server enforces per-alert restrictions. The prompt cannot reliably enforce either.

The driver uses the `command`, `args` and `env` from your `mcp.json` entry to launch:

```
openshell sandbox exec -n <sandbox>
--no-login-shell --no-tty [--env KEY=VALUE ...]
-- <command> <args...>
```

- **Transport.** The driver speaks JSON-RPC over the process's standard input and output. The first byte written to standard output must be `{`. Banners and log lines break the stream; log to standard error.
- **Fresh sandbox.** The driver starts the server with no staged files. Specify the root that your server serves in `args` or `env`. The example in `examples/mcp.example.json` does not specify a root. If your server analyzes a staged copy (Section 9.1), create that copy and its workspace at startup.
- **Initialization.** Respond to `initialize` immediately. Initial CLDK analysis can take minutes, and the driver limits how long it waits for the first response byte. Build the analysis on the first analysis call or in the background. Staff will call your analysis tools on a server started this way.

Where each attempt stops

Figure 4 shows the request path and three attempts:

- **Planted text:** the model may follow the injected instructions. The MCP server must refuse any resulting forbidden call.
- **Raw MCP call:** the server refuses the forbidden operation.
- **Sandbox process:** the kernel blocks access to the answer key and writes outside the workspace, but permits reads of another case.

The code-action agent has no server layer.

7 Design report and comparisons

Your design report must address Part A of the rubric and include two comparisons:

1. **Code-action agent versus MCP agent.** Run both agents on the same fixed OWASP alert list, using the same model and budget. Compare model turns, context tokens, tool calls by group, labels and their agreement with the answer key, and patches accepted by the three checks. Explain where each agent's access rules are enforced.
   Relate your results to the design tradeoff from lecture 4, slide 154. A script can make several decisions within one model turn and return a summary, reducing the need for additional turns. An MCP server can validate and refuse each structured tool call. A script's internal operations require restrictions below the Python interpreter.
2. **Direct tool calls versus MCP calls.** Choose an operation supported by both your HW1 and HW2 implementations, such as reading a file. Trace it through the HW1 path (model request, dispatcher, in-process tool) and the HW2 path

(model request, dispatcher, MCP client, server, tool). Use your own code for both paths; the tool names need not match. Explain tool discovery, contracts and validation, serialization, where each component runs, what it can access, and how errors propagate. Include one malformed-argument case at each layer.

8 Rubric

The assignment is marked out of 19, one point per item, scaled to the assignment's 20 percent of the course grade. These items assess the implementation of the required mechanisms and the documentation and evidence that demonstrate them, not the correctness or performance of classifications.

Part A: design report (5)	
A1	**Tool contracts** for the tools you choose to expose: arguments, results, refusal cases, and what the description does not enforce.
A2	**The call path** of one tool through host, client, server and backend, with where discovery, validation and serialization happen.
A3	**Access rules:** state the read, write, and network rules. Identify the fixed and per-alert restrictions, where each is enforced, and how each enforcement layer could be bypassed (lecture 6).
A4	**Sandbox policy:** explain every permission granted or withheld.
A5	**Patch acceptance:** the three checks, and what they do not establish.

Part B: implementation (9)	
B1	**The server** exposes CLDK analysis through tools of your choice and enforces their argument and result schemas.
B2	**The code-action agent:** clone the CLDK Python SDK and use Claude Code or Codex to write the agent's prompt file. The harness loads that prompt, and the agent initializes CLDK and composes tools using either heredoc scripts or a Python execution kernel you implement. Both run inside the sandbox.
B3	**Editing and patch acceptance:** both agents can edit workspace files, rescan them with Bandit, run the three checks, and record the diff in `report.json`. Tool names and interfaces are your choice.
B4	**Discovery and the model's tool list** derived from `tools/list`, minus the host-only tools, recorded as a `discovery` event.
B5	**Dispatcher refusals** for unknown tools, bad arguments and host-only tools.
B6	**Runtime-controlled remediation:** the runtime sends only alerts labelled 1 to remediation and skips all others. The trace records each decision as a `routing` event with `by: runtime`.
B7	**Per-alert restrictions:** the server enforces read and write rules through `set_alert` and records `refused_by` on refusals. The MCP agent's own process never opens a target file.
B8	**The sandbox:** both agents and the server run inside OpenShell with `hard_requirement` Landlock, no grant on the target root, the workspace the only writable target path.
B9	**The network rule:** only the model endpoint, and the provider key never present in the sandbox environment.

Part C: evidence and README (5)	
C1	**README.** From a fresh clone we can build the sandbox, start either agent inside it, and the staff driver finds the server from `mcp.json`.
C2	**The live-run measurement** under the planted injection, for both agents, with trace lines.
C3	**The two comparisons** of Section 7.
C4	**Limitations.**
C5	**Prompt used to create the CLDK prompt file.** Submit the exact prompt you gave Claude Code or Codex to generate your agent's CLDK prompt file. This is separate from the generated file that the agent loads.

Grading notes:

- Grades assess implementation and the required supporting artifacts, including the generation prompt in C5. Accuracy, precision and recall do not affect grades.
- A missing required mechanism or artifact cannot earn its point.
- We check B2 on OWASP BenchmarkPython.

- Staff will build the sandbox using your README and run the staff driver (Section 6). We will compare your analysis tools' answers with the corresponding staff-run CLDK queries on OWASP. We will then run the MCP agent on all three targets and the code-action agent on OWASP, inside the sandbox with the full configuration. We will inspect the discovery, routing, and remediation events from one run of each agent, as applicable.

9 README

Your `README.md` must provide enough detail for staff to build the sandbox and run both agents from a fresh clone without further help. You may choose your own installation and launch procedure. Document the following:

1. **Setup commands.** Include every command needed to install Python 3.12, Bandit 1.9.4, CLDK 2.0.0, the MCP SDK, and your other dependencies. List the environment variables your agents read, using names only, never values.
2. **CodeAct prompt.** Explain how to clone the CLDK Python SDK and use Claude Code or Codex to write the prompt file. Give the paths to both the generation prompt you submitted to that assistant and the resulting agent prompt file. Explain how your harness loads the agent prompt. It must instruct the agent to initialize CLDK on its workspace and compose tools using either heredoc scripts or your Python execution kernel.
3. **MCP tool contracts.** Document the tools you expose, including their names, arguments, results, and refusal cases. Identify the tools that provide analysis, source access, editing, and rescanning so staff can test the required behavior using your interfaces.
4. **Sandbox build.** Give the command to build the image from your repository, with the target at `/target` (Section 9.1).
5. **Launch commands.** Give one command for each agent. Each must create or reuse a sandbox based on `policy.yaml` and run your agent inside it with `--input`, `--output`, and `--config`, as specified in Section 5. Explain how to retrieve the output files from the sandbox.
6. **Configuration files.** Commit `configs/mcp.toml` and `configs/codeact.toml`.
   Both must validate against `config.schema.json`, use the OWASP `[scan]` block, and include a budget you consider sufficient for a complete run. They must be identical except for `[agent] kind`. For MCP runs on Radicale and SGLang, staff will replace only the `[scan]` block with the required block for that target.

Staff will run the MCP agent on each target, the code-action agent on OWASP, and the staff driver. Every agent run must produce `report.json` and `trace.jsonl`.

9.1 The sandbox image and the launch commands

Platform. OpenShell supports Linux, macOS on Apple Silicon, and Windows with WSL 2. It also requires Docker, Podman, or host virtualization. Landlock ABI v3 requires Linux kernel 6.2 or newer on the sandbox host.

If you use an Intel Mac or Windows without WSL 2, run Ubuntu 24.04 in a Linux VM using Multipass or UTM. Install Docker and OpenShell inside the VM, following the Linux setup procedure. Ask the instructor if you need help.

Staff tested OpenShell 0.1.4 only on macOS with Apple Silicon and Colima. Staff have not tested the VM procedure, Docker Desktop, or the other platforms. Read the project's `install.sh` before using it to install OpenShell:

`https://raw.githubusercontent.com/NVIDIA/OpenShell/main/install.sh`

If you use Colima, write the complete configuration below before starting a sandbox. This replaces the service's configuration file. Set `socket_path` to the path reported by `docker context inspect`, without the `unix://` prefix. Then restart the gateway with the command in the first line.

```
# ~/.config/openshell/gateway.toml, then: brew services restart nvidia/openshell/openshell
[openshell]
version = 2
[openshell.gateway]
compute_driver = "docker"
[openshell.drivers.docker]
socket_path = "/Users/<you>/.colima/default/docker.sock"
grpc_endpoint = "https://host.docker.internal:17670"
```

If sandbox creation fails with `ControlSupervisorStartFailed`, check for a missing `grpc_endpoint`. The gateway also requires the first three settings in the example. OpenShell normally finds Docker Desktop's socket automatically. If Docker Desktop produces the same error, create this configuration file using its socket path.

Sandbox image. The `sandbox create --from` option requires an image reference. Build the image with the gateway's container engine before creating the sandbox. The staff image used:

- `python:3.12-slim` as the base image;
- the agent, server, and pinned dependencies under `/app`, installed during the build because the sandbox cannot access a package index;
- the pinned OWASP checkout at `/target`, owned by root and readable by all users, so that access denials came from the sandbox policy;
- a non-root user with UID 1000 who owned `/sandbox`.

Build one image per target, with that target at `/target`. Your `policy.yaml` can list the code directories of all three targets; OpenShell skips listed paths that do not exist in an image.

Input workspace. The policy does not grant access to `/target` itself. An agent that copies its input directory therefore cannot use `/target` as `--input`. Copy only the permitted entries to `/sandbox/work/target` and pass that directory as `--input`. Exclude the answer key and `.git`.

Run CLDK on this workspace copy as well. On `/target`, CLDK cannot create its project cache, `.codeanalyzer`. Even if the cache is moved, CLDK cannot list `/target`, so it returns no callables without raising an error.

Model provider. For Anthropic, edit the top-level `binaries` list in OpenShell's `providers/anthropic.yaml` to name your Python interpreter. Import the edited profile, create the provider using `ANTHROPIC_API_KEY` from your shell, and attach it when creating the sandbox:

```
openshell provider profile import -f anthropic.yaml --global
openshell provider create --name anthropic --type anthropic --from-existing
openshell sandbox create --name <name> --from <image> --policy policy.yaml \
   --provider anthropic --detach -- sleep infinity
```

Inside the sandbox, the environment variable contains a placeholder; the actual key remains outside. The attachment adds a second network policy, `_provider_anthropic`, with the profile's endpoint and binaries. Explain the permissions from both network policies in your design report.

For an OpenAI-compatible model server on your own machine, use `examples/policy.local.yaml`. It specifies a `tcp` endpoint using a hostname for your machine. OpenShell rejects IP literals and its reserved aliases: `host.docker.internal`, `host.containers.internal`, and `host.openshell.internal`. On Colima, use `host.lima.internal`.

Example launch. Staff used the following commands on Colima, with the OWASP checkout in the image and a model server on the Mac. This is an abridged example: the configuration was first written to `/sandbox/work/low.toml`, and `your-agent` refers to the HW1 agent.

```
openshell sandbox create --name hw2local --from hw2-feasibility:1 --policy policy.local.yaml \
   --no-auto-providers --detach -- sleep infinity
openshell sandbox exec -n hw2local --no-login-shell --no-tty \
   --env LOCAL_MODEL_URL=http://host.lima.internal:8080 -- sh -c '
  mkdir -p /sandbox/work/target &&
  cp -R /target/testcode /target/helpers /target/app.py /sandbox/work/target/ &&
  cd /app/agent &&
  your-agent --input /sandbox/work/target --output /sandbox/work/run4 --config /sandbox/work/low.toml'
```

During development, you may upload the target instead of embedding it in the image. For OWASP, create a staging directory named `target` containing only `testcode/`, `helpers/`, and `app.py`. Upload it with `openshell sandbox upload <name> <staging>/target /sandbox`. Never upload the full checkout: it contains the answer key, and `/sandbox` is writable.

10 Deliverables

- All source code, including the server, and a lockfile or equivalent that pins dependencies.
- The CodeAct prompt file written with Claude Code or Codex, reviewed by you and loaded by your agent's harness.
- The exact prompt you gave Claude Code or Codex to create that CLDK prompt file, submitted as a separate text or Markdown file (rubric item C5, one point).
- `README.md` as Section 9 describes, with `configs/mcp.toml` and `configs/codeact.toml`.
- `mcp.json` and `policy.yaml` at the repository root.
- The design report (Part A plus C2, C3 and C4), as `REPORT.md` or a PDF.
- The unedited `report.json`, `trace.jsonl`, and configuration for every run cited in your design report. Do not include API keys in the repository. Do not include OpenShell log exports or per-cell traces; staff will run the agents and driver directly.
- A contribution statement that says who did what.

11 Submissions

We will create the HW2 assignment on CourseWorks (Canvas). Submit there and follow the instructions on the assignment page. Each team submits once; any member may submit on behalf of the team.

12 Academic integrity and AI tools

You must use Claude Code or Codex to help write the CodeAct prompt file, as specified in Section 2.1. This use of AI assistance is part of the assignment. Review the prompt and verify that it directs your agent to initialize CLDK and compose tools using your chosen execution method: heredocs or a Python execution kernel.

You may also use AI assistants to learn, debug your own code, review a design, or complete small sections of code through autocomplete. Do not ask an agent to complete the entire assignment and submit its output. You are responsible for every line you submit. Every team member must be able to explain the entire submission; the individual exams will test this understanding.

Do not use outside AI assistants to generate or modify the target code. This restriction does not apply to the agents you build for this assignment: they must modify the workspace copy during the required remediation stage. The course's AI-use policy is strictly enforced.